Raport SIGK 3

Nel Kułakowska, Zuzanna Jacyna

Architektura

W ramach projektu zaimplementowano 2 architektury

1. Deterministyczną (bez szumu) – gdzie wejściem jest wyłącznie wektor 14 parametrów sceny.
2. Generatywną (z szumem) – gdzie wejście stanowi połączenie parametrów sceny oraz 100-wymiarowego wektora szumu.

Architektura Generatora

1. Wejście: Wektor parametrów sceny (14)
2. Warstwa w pełni połączona (Linear) 14 wejść -> 512 * 4 * 4 wyjść
3. Zmiana kształtu (Reshape): (512, 4x4)
4. Blok ConvTranspose2d (512 -> 256), BatchNorm2d, ReLU
5. Blok ConvTranspose2d (256 -> 128), BatchNorm2d, ReLU
6. Blok ConvTranspose2d (128 -> 64), BatchNorm2d, ReLU
7. Blok ConvTranspose2d (64 -> 32), BatchNorm2d, ReLU
8. Blok ConvTranspose2d (32 -> 3), Tanh
9. Wyjście: Obraz RGB (3, 128x128)

Architektura Dyskryminatora

1. Wejście: Obraz RGB (3, 128x128) oraz Parametry sceny (14)
2. Parametry sceny -> Rozszerzenie do (14, 128x128)
3. Połączenie obrazu i parametrów sceny, kształt po połączeniu: (17, 128x128)
4. Conv2d (17 -> 64), LeakyReLU
5. Conv2d (64 -> 128), BatchNorm2d , LeakyReLU
6. Conv2d (128 -> 256), BatchNorm2d , LeakyReLU
7. Conv2d (256 -> 512), BatchNorm2d , LeakyReLU
8. Conv2d (512 -> 1024), BatchNorm2d , LeakyReLU
9. Wypłaszczenie (Flatten)
10. Warstwa w pełni połączona (Linear) 1024 * 4 * 4 wejść -> 1 wyjście
11. Sigmoid

Architektura Generatora - wersja z szumem

Na wejściu zamiast podać jedynie parametry wektora sceny dodajemy losowy szum, zwiększając rozmiar wejścia o rozmiar szumu.

Funkcja Straty

Dla generatora przyjęto złożoną funkcję straty:
*BCE Loss + 100 * L1 +10 * LPIPS*
BCE Loss odpowiada ona za "oszukiwanie" Dyskryminatora. Zastosowano L1 zamiast jak zazwyczaj L2, ponieważ sprzyja to ostrzejszym obrazom. Składnik LPIPS został zastosowany aby poprawić jakość detali, których nie zauważa L1.

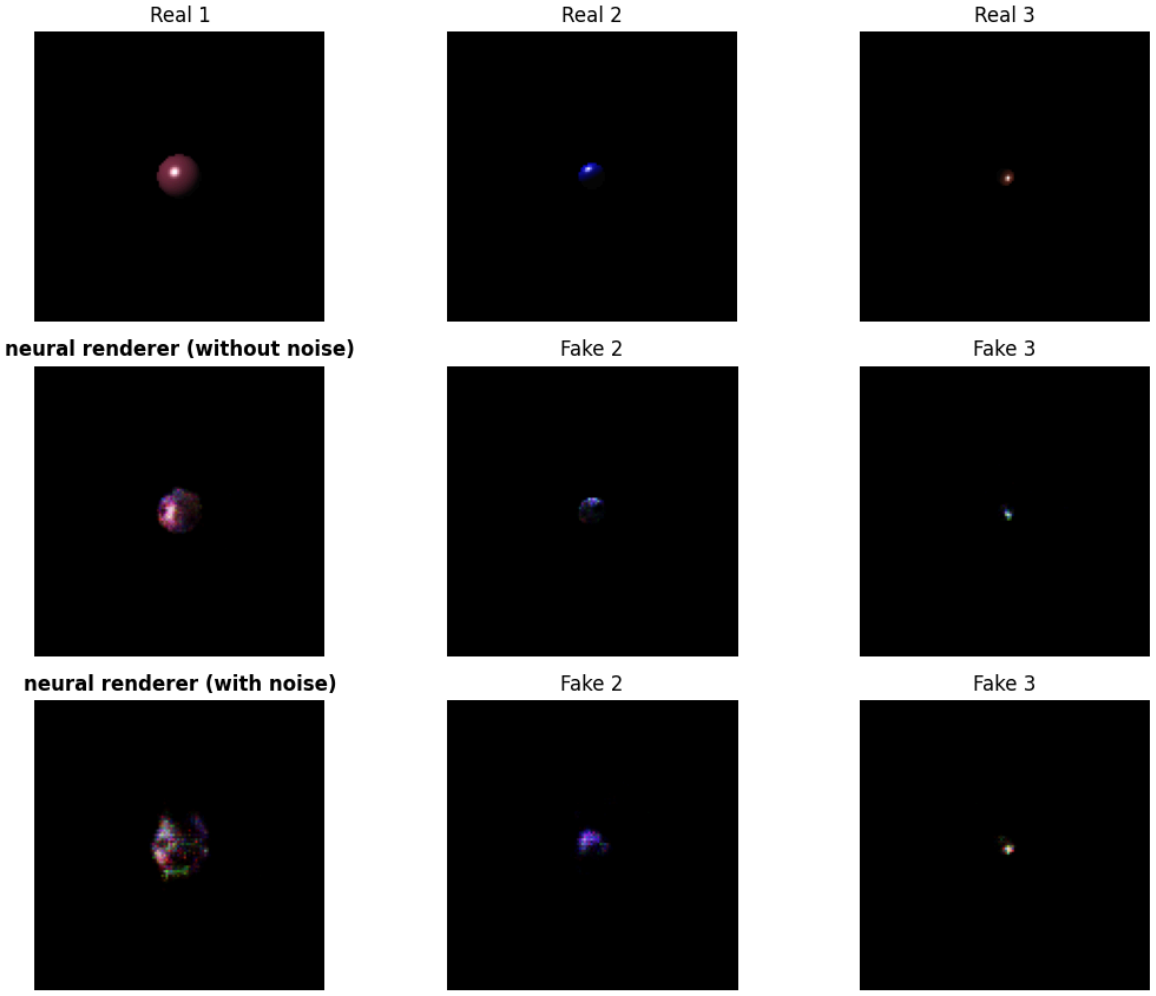
Dla dyskryminatora zastosowano BCE Loss, obliczając średnią arytmetyczną z błędów dla dwóch przypadków: obrazów fałszywych i prawdziwych.

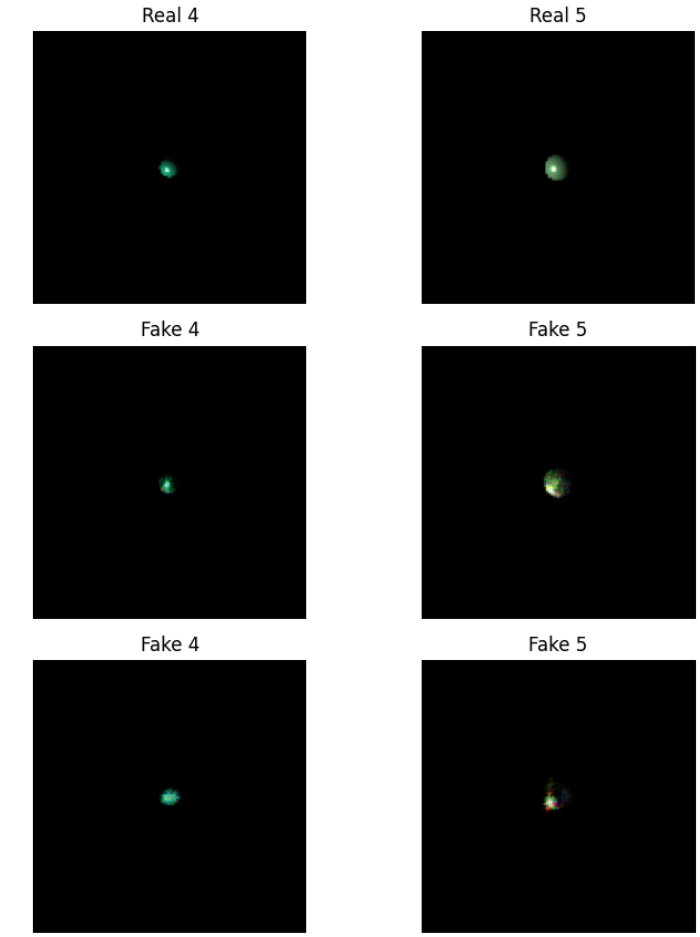
Rezultaty

Tabela z wynikami metryk

Metoda	FLIP	LPIPS	SSIM	Hausdorff
neural renderer (without noise)	0.0175	0.0534	0.9876	3.16
neural renderer (with noise)	0.0227	0.0834	0.9646	6.25

Przykładowe obrazki





Czy wszystkie metryki dobrze oddają jakość generowanych obrazów?

Nie, nie każda metryka w równym stopniu oddaje rzeczywistą jakość wygenerowanego obrazu.

- FLIP - Bardzo dobrze oddaje jakość z punktu widzenia ludzkiego oka
- LPIPS - Wyłapuje różnice w teksturze i rozkładzie barw, których nie łapią proste metryki pikselowe.
- SSIM - metryka ta w kontekście renderingu bywa nieco myląca. SSIM ocenia ogólną strukturę i kontrast. Może zwracać bardzo wysokie wyniki (>0.90) nawet wtedy, gdy brakuje bardzo ważnego, ale małego elementu.
- Odległość Hausdorffa - Ta metryka najgorzej oddaje ogólną wizualną jakość obrazu. Wystarczy kilka zaszumionych pikseli w pobliżu krawędzi, a detektor uzna to za nową krawędź i drastycznie podbije wynik błędu.
- Ludzkie oko - potrafi lepiej ocenić niż większość metryk.

Wnioski

Bardzo dobrym pomysłem okazało się “rozgrzanie” generatora, aby mógł się nauczyć układu i zarysu sceny bez kar ze strony Dyskryminatora.
Co można by było poprawić:

- Architektura U-Net - zastosowanie *skip-connections* aby modele mogły lepiej nauczyć się krawędzi.
- Więcej parametrów sceny (szczególnie związanych ze światłem).
- Bardziej skomplikowana/lepiej przetestowana funkcja celu.